

Fast and the Furious: Hot Starts in Pursuit-Evasion Games

Gabriel Smithline
University of Michigan
Ann Arbor, MI, USA
gsmithl@umich.edu

Scott Nivison
Air Force Research Laboratory
Shalimar, FL, USA
scott.nivison.1@us.af.mil

1 Introduction

Pursuit-evasion games model interactions between a set of pursuers and evaders in differential games. These games are critically important in fields such as control, aerospace, and robotics, and form the foundation of adversarial optimal control [15, 35]. Since their inception in the 1950s and 1960s, pursuit-evasion games have seen significant advancements, with one of the most foundational contributions being the Hamilton-Jacobi-Isaacs (HJI) equation [15] introduced by Rufus Isaacs. The HJI equation, a generalization of the Hamilton-Jacobi-Bellman (HJB) equation [4], extends to zero-sum games where one player aims to minimize a cost function while another aims to maximize it. In these games, the optimal strategies for pursuers and evaders must satisfy the HJI. The equation, a partial differential equation (PDE), captures the game’s dynamics, including player positions, velocities, and optimal control actions. The solution to the HJI equation, known as the value function, represents the minimum cost for pursuers to capture evaders, considering the evaders’ optimal responses.

Despite these advancements, many open-ended questions remain. A significant challenge is strategically placing pursuers without knowing the number or locations of evaders. Developing methods to address this issue could enhance algorithm effectiveness and efficiency, thereby reducing collective pursuer costs.

In this paper, a novel approach is proposed that leverages artificial intelligence and machine learning techniques to offline generate initial configurations for pursuers before evaders appear, a strategy termed *hot starts*. Inspired by recent advances in Graph Neural Networks (GNNs) [31], particularly the methodology described in Google DeepMind’s TacticAI developed for Liverpool FC [34], we aim to apply similar methods to pursuit-evasion games. Two pure pursuit game scenarios are considered: One Evader vs. Many Pursuers and Many Evaders vs. Many Pursuers.

Our method demonstrates enhancements by not making direct assumptions about evader positions, quantities, algorithms, optimization issues, and state-space simplifications with these hot starts. To overcome the lack of domain-specific expert feedback on configurations, games are simulated using the Multi-Agent Particle Environment (MPE) [24, 27], a flexible simulation framework designed for multi-agent interactions (see Figure 3). Through these simulations, empirical data is generated, which serves two purposes: first, it provides the data needed to train the Graph Convolutional Network (GCN) [20] to extrapolate patterns; second, it allows us to construct a Graph Feature Space (GFS) using three features: U_{capture} (capture potential), U_{distance} (distance), and U_{heading} (heading angle). Survival analysis and log-rank tests are performed to empirically evaluate whether hot starts have a statistically significant impact. Key contributions are outlined as follows:

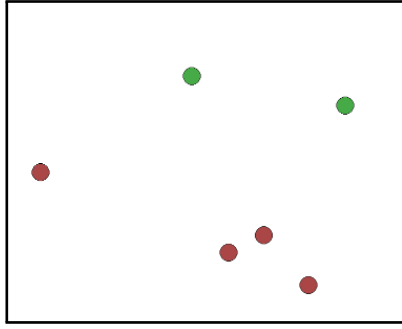
- Encode pursuer configurations in the GFS, allowing us to solve a multi-objective optimization problem offline and construct a Pareto frontier of pursuer configurations.
- Train the GCN using a loss function designed to learn from the Pareto-valued configurations. Track hypervolume and generational distance metrics to measure how the GCN learns the Pareto front and how well it generalizes by measuring the volume of the frontier it covers.
- Inspired by TacticAI, generate numerous configurations with a GCN and use statistical methods to evaluate the impact of these configurations on pursuer efficiency and evader capture.

2 Related Work

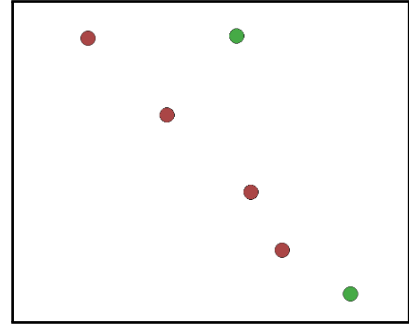
To model effective gameplay in pursuit-evasion games, algorithms often utilize Apollonius circles [11, 15, 29], and strategies based on Voronoi diagrams or Delaunay triangulation [3, 33] (for a more thorough overview on differential games, see Appendix Section A). Previous works have explored simplified, discretized environments where pursuers are explicitly assigned to evaders based on probabilistic estimation techniques, or where the explicit algorithms of the adversary are known by each party [2, 12, 19, 37]. Other studies highlight redundant allocation of pursuers [33], effectively increasing the cost of pursuit for the collective of pursuers.

Recent works have extended the application of graph-based learning techniques to multi-agent systems in discrete environments. For instance, Li et al. [21, 22] structured agent interactions within a graph to solve pursuit-evasion games, with a focus on real-time coordination and task assignment. While effective for discrete settings, these methods do not address the dynamic and continuous aspects of real-world environments, nor do they tackle the same problem focused on in this work. This work diverges by introducing a novel process for generating "hot starts" without prior knowledge of evader positions. By leveraging differential game theory and operating in continuous space, time-dependent algorithms are accounted for, for both pursuers and evaders, which contrasts with the real-time updates used in discrete environments. This multi-objective optimization approach allows us to generate Pareto-optimal configurations that improve pursuer efficiency from the outset. The combination of continuous space, differential dynamics, and the distinct focus on initial agent allocation sets this work apart in the domain of pursuit-evasion games, making it applicable to a wider range of real-world scenarios.

The concept of initial agent allocation in large-scale systems has also been explored in particle swarm optimization and genetic algorithms, particularly in population initialization, where the goal is to initialize the population advantageously [17]. Traditionally, evolutionary algorithms use pseudo-random number generators



(a) 4 Pursuers vs. 2 Evaders Random Start



(b) 4 Pursuers vs. 2 Evaders Hot Start

Figure 1: Images of a hot start and a random start in 4 Pursuer vs. 2 Evader Game (pursuers are in red and evaders are in green).

for population initialization; more recent work explores intelligent initialization through AI-based heuristics [18].

While GNN-based approaches like those used in Google DeepMind’s TacticAI for sports strategy [34] have shown success in generating player configurations for specific tasks (e.g., corner kicks), applying similar methods to pursuit-evasion games poses distinct challenges. In sports, configurations benefit from domain-specific expertise and clear objectives, such as the positioning of players based on real-time feedback from coaches. By contrast, pursuit-evasion games often lack this level of expert guidance, and there is the added complexity of not knowing evader positions or quantities in advance.

Previous works have not fully explored the challenge of strategically positioning pursuers without such prior knowledge, which is a fundamental aspect of real-world pursuit-evasion scenarios. This approach seeks to explore this gap by leveraging GNNs to generate initial configurations, termed “hot starts,” without making direct assumptions about the number or location of evaders. This method has potential applications in a wide range of domains, including UAV search and rescue [9], wildlife protection [7, 36], human-computer interaction [23], traffic management [25], and general security [1, 32], where such strategic initialization could significantly enhance operational efficiency.

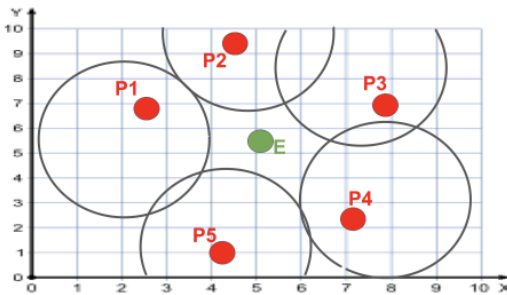


Figure 2: Image of 5 pursuers (in red) surrounding an evader (in green), illustrating cooperative containment. Black circles represent capture radii.

3 Control Algorithms

3.1 1 Evader vs. Many Pursuers

Control algorithms using heuristics inspired by strategies from [10, 11] were adapted for our scenarios, taking into account the varying speed ratios between evaders and pursuers. In the literature, different speed ratios lead to distinct geometric constructs: Apollonius circles or Cartesian ovals when $\gamma \neq 1$, and perpendicular bisectors or hyperbolas when $\gamma = 1$. In our work, for the 1 evader vs. many pursuers scenario, we assume $\gamma = \frac{v_E}{v_P} > 1$, where the evader is faster than the pursuers. Here, the evader identifies the weakest link between pursuers to escape, while pursuers aim to contain the evader strategically. (See Appendix B for detailed algorithms and system dynamics.)

3.2 Many Evaders vs. Many Pursuers

For many-vs-many scenarios ($\gamma = \frac{1}{2} < 1$), where pursuers are faster than evaders, we employ a hybrid approach combining geometry-based controls with PDE solutions. Specifically, we use a first-order upwind finite difference scheme with directional bias: backward differences are selected for negative gradients and forward differences for positive gradients. This scheme provides numerical stability while respecting the information flow direction in the HJI equation, consistent with methods described in [26, 28]. The Hamiltonian computation explicitly accounts for the minimizing pursuer and maximizing evader roles, ensuring appropriate upwind direction selection based on agent type.

Our hybrid approach uses a weighted parameter $\alpha = 0.5$ to combine geometric distance metrics with PDE value function information in the assignment cost matrix for the Hungarian algorithm. The geometric measures specifically include: (1) Euclidean distances between pursuers and evaders for the assignment cost matrix, (2) normalized direction vectors for determining movement trajectories, (3) relative bearing angles for optimal interception paths, and (4) proximity-based avoidance vectors that allow evaders to compute escape directions from nearby pursuers. Pursuers are allocated to evaders by minimizing this hybrid cost, and control actions are computed using both gradient information from the PDE and direct geometric vectors, combining them as:

$$\mathbf{u}_{\text{final}} = \alpha \cdot \mathbf{u}_{\text{geometric}} + (1 - \alpha) \cdot \mathbf{u}_{\text{gradient}} \quad (1)$$

where $\mathbf{u}_{\text{geometric}}$ is derived from the instantaneous geometric relationship between agents, and $\mathbf{u}_{\text{gradient}}$ comes from the spatial derivatives of the value function. Evaders employ escape strategies based on gradient ascent of the value function combined with geometric avoidance of nearby pursuers, computing avoidance vectors that are inversely proportional to the squared distance from approaching pursuers when they enter a predefined avoidance radius.

Our numerical approach, while using discrete time steps, aims to approximate continuous-time game dynamics. The qualitative results align with theoretical predictions for pursuit-evasion scenarios with the given speed ratios [5, 14, 35], demonstrating the efficacy of our implementation despite the inherent limitations of numerical approximation.

4 Graph Feature Space

4.1 Graph Features

The Graph Feature Space (GFS) is an objective space where configurations of agents are mapped based on certain features, sampled during our data-generation phase. The three features we focus on to construct the Pareto frontier are: U_{capture} (capture potential), U_{distance} (distance), and U_{heading} (heading angle). These features are computed offline for each pursuer within the PettingZoo environment, as illustrated in Figure 3. We delve into their definitions below.

4.2 Pareto Frontier Definition

The Pareto frontier is formally defined as the subset $\Pi_p \subset \Pi$, where Π is the set of all possible configurations. Formally, Π_p is found such that:

$$\Pi_p = \{G_i \in \Pi \mid \nexists G_j \in \Pi \text{ such that } \hat{c}_{G_j} \succ \hat{c}_{G_i}\}, \quad (2)$$

where $\hat{c}_{G_i}$ is the vector containing our three features of the GFS, for configuration G_i , and $\hat{c}_{G_j} \succ \hat{c}_{G_i}$ indicates that the feature vector of G_j strictly dominates that of G_i . More precisely, for G_j to strictly dominate G_i :

$$\begin{aligned} \hat{c}_{G_j} \succ \hat{c}_{G_i} &\iff \forall k \in \{1, \dots, n\}, \hat{c}_{G_j,k} \geq \hat{c}_{G_i,k} \\ &\wedge \exists l \in \{1, \dots, n\}, \hat{c}_{G_j,l} > \hat{c}_{G_i,l} \end{aligned} \quad (3)$$

where n is the number of objectives. The set Π_p thus represents the Pareto-optimal configurations, where no configuration in Π strictly dominates any element of Π_p [38].

4.3 Feature Definitions

Specifically, U_{capture} is expressed as:

$$U_{\text{capture}} = 2 \left(1 - \frac{1}{1 + e^{\frac{-d_i + r}{r}}} \right) \quad (4)$$

where:

- d_i is the distance between the pursuer and the i -th evader.
- r is the capture radius.
- n is the number of evaders.

This utility is designed to be 1 when the distance $d_i = r$, and approaches 0 as the distance d_i increases. When $d_i < r$, $U_{\text{capture}} > 1$, indicating a higher capture potential within the capture radius. The sigmoid component smoothly transitions the utility value, avoiding

abrupt changes and adding stability to optimization and decision-making processes.

U_{distance} is expressed as:

$$U_{\text{distance}} = \min_{i=1, \dots, n} \|\mathbf{p} - \mathbf{e}_i\| \quad (5)$$

where:

- $\mathbf{p}$ is the position vector of the pursuer.
- $\mathbf{e}_i$ is the position vector of the i -th evader.
- n is the number of evaders.

This utility represents the shortest distance between the pursuer and any of the evaders. It focuses on the closest evader, which is critical in pursuit-evasion games where the goal is to minimize the distance to capture the nearest evader.

Finally, U_{heading} is expressed as:

$$U_{\text{heading}} = \frac{1}{n} \sum_{i=1}^n \arccos \left(\frac{\mathbf{v}_p \cdot (\mathbf{e}_i - \mathbf{p})}{\|\mathbf{v}_p\| \|\mathbf{e}_i - \mathbf{p}\| + \epsilon} \right) \quad (6)$$

where:

- $\mathbf{v}_p$ is the velocity vector of the pursuer.
- $\mathbf{p}$ is the position vector of the pursuer.
- $\mathbf{e}_i$ is the position vector of the i -th evader.
- n is the number of evaders.
- $\epsilon = 10^{-9}$ is a small constant to prevent division by zero.

This utility measures the alignment between the pursuer's heading direction and the directions to each evader. Lower values of U_{heading} (i.e., smaller angles) indicate better alignment between a pursuer's heading and the direction to an evader when locations are known. In scenarios where evader locations are unknown, maximizing U_{heading} (achieved by averaging) encourages pursuers to consider a wider range of heading angles, increasing the likelihood that pursuers are heading in beneficial directions when evaders appear.

Illustrative examples demonstrating the functionality of U_{heading} and U_{capture} are provided in Section D of the Appendix.

4.4 Normalization and Aggregation

To ensure comparability among the different features and maintain the sum-to-one constraint, we normalize the aggregated features proportionally. Specifically, for each configuration G_i , we aggregate the utilities by averaging over all pursuers:

$$U_k(G_i) = \frac{1}{N_p} \sum_{j=1}^{N_p} U_k^{(j)}, \quad k \in \{\text{capture, distance, heading}\}$$

where N_p is the number of pursuers in the configuration G_i , and $U_k^{(j)}$ is the feature for pursuer j .

To enforce the sum-to-one constraint across all utilities, each aggregated utility $U_k(G_i)$ is normalized by the total sum of all three utilities for the configuration:

$$\tilde{U}_k(G_i) = \frac{U_k(G_i)}{\sum_{m=1}^3 U_m(G_i)}$$

This ensures that:

$$\sum_{k=1}^3 \tilde{U}_k(G_i) = 1$$

By applying this proportional normalization, we maintain a balanced contribution of each utility feature in the multi-objective

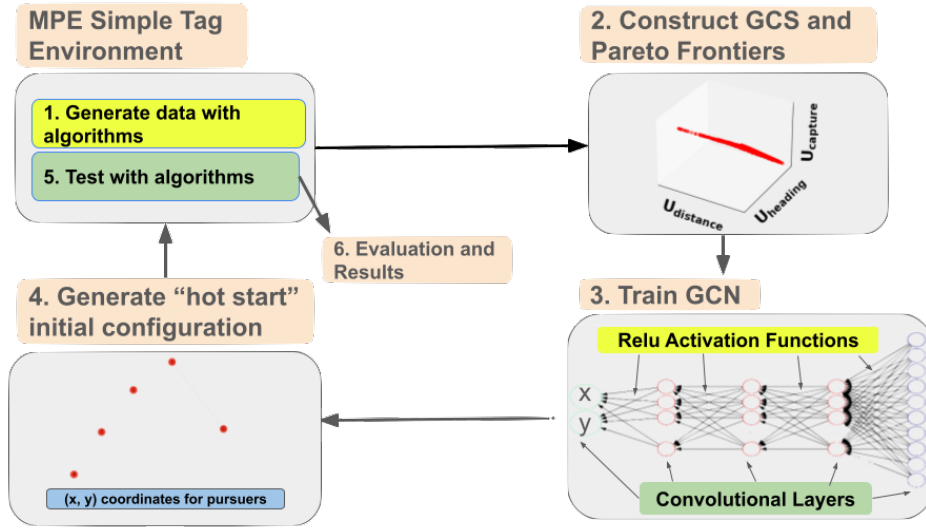


Figure 3: Pipeline diagram of the Pursuit-Evasion Games framework. Starting from the MPE Simple Tag Environment, the process follows these steps: 1. Generate data using control algorithms; 2. Construct the Graph Feature Space (GFS) and identify Pareto frontiers; 3. Train the Graph Convolutional Network (GCN); 4. Generate Hot Starts (each red dot represents x and y coordinates); 5. Apply these Hot Starts to the control algorithm; and 6. Perform statistical evaluation.

optimization process. This facilitates a fair and consistent comparison of pursuer configurations, ensuring that no single feature disproportionately influences the optimization outcomes.

4.5 Construction of the Graph Feature Space (GFS)

To construct the GFS and identify Pareto-optimal pursuer configurations, we formulate a constrained multi-objective optimization problem. We utilize the NSGA-II genetic algorithm [8], with a population size of 500 and 200 generations, to solve this problem. We start by generating a diverse set of pursuer configurations. Each configuration G_i consists of multiple pursuers positioned in the environment. These configurations are generated through simulations or sampling methods and are stored in the set Π .

For each configuration $G_i \in \Pi$, we compute the three normalized and aggregated features:

$$\tilde{U}_k(G_i) = \frac{U_k(G_i)}{\sum_{m=1}^3 U_m(G_i)}, \quad k \in \{\text{capture, distance, heading}\}$$

Our goal is to identify configurations that offer the best trade-offs among the three features. We formulate the multi-objective optimization problem as:

$$\underset{G_i \in \Pi}{\text{minimize}} \quad \mathbf{f}(G_i) = [-\tilde{U}_{\text{capture}}(G_i), \tilde{U}_{\text{distance}}(G_i), -\tilde{U}_{\text{heading}}(G_i)] \quad (7)$$

The objectives are:

- **Maximize $\tilde{U}_{\text{capture}}(G_i)$:** Increase the potential effectiveness in capturing evaders.
- **Minimize $\tilde{U}_{\text{distance}}(G_i)$:** Reduce the average minimum distance to evaders.

- **Maximize $\tilde{U}_{\text{heading}}(G_i)$:** Encourage pursuers to have diverse heading directions.

Since standard optimization algorithms perform minimization, we negate the objectives we wish to maximize. The proportional normalization ensures that each feature is appropriately scaled, facilitating a balanced optimization process.

From the Pareto-optimal configurations determined by our optimization problem, we construct graphs by adding edges between pursuer nodes called PP Edges.

4.5.1 Pursuer-Pursuer (PP) Edges. A PP edge exists between two pursuers if the distance between them is less than or equal to the average distance between all pairs of pursuers in the configuration. The weight of a PP edge is defined as:

$$w_{PP} = \frac{1}{d_{PP}} \quad (8)$$

where d_{PP} is the distance between the two pursuers.

Each node in the graph consists of the following features: $\tilde{U}_{\text{capture}}$, $\tilde{U}_{\text{distance}}$, $\tilde{U}_{\text{heading}}$, capture radius, x & y velocities, and x & y positions.

We employ the NSGA-II algorithm to solve this optimization problem. The algorithm operates as follows:

- (1) **Decision Variables:** The configurations G_i in Π .
- (2) **Evaluation:** For each configuration G_i , compute the normalized features and evaluate the objective vector $\mathbf{f}(G_i)$.
- (3) **Constraint Handling:** Enforce the constraint that the sum of the features equals 1.
- (4) **Selection:** Identify non-dominated configurations to form the Pareto front.

The result of the optimization is a set of Pareto-optimal configurations $\Pi_p \subset \Pi$. These configurations represent the best trade-offs among the objectives, where no other configuration in Π is strictly better in all objectives.

5 Graph Convolutional Network

To generate our hot starts, we use a Graph Convolutional Network (GCN) trained on the graphs from the GFS that fall on the Pareto frontier. GCNs are advantageous for operating on graph-structured data, making them suitable for tasks like node classification, link prediction, and graph generation. The GCN follows a structure where it processes the features of pursuer nodes and their connections to output optimized configurations.

5.1 Pareto Loss

During training, we monitor several performance measures such as Hypervolume, Generational Distance (GD), Generational Distance Plus (GD+), Inverted Generational Distance (IGD), and Inverted Generational Distance Plus (IGD+) [6, 39]. Our primary loss function, called Pareto loss, is defined to measure the distance between the utility feature vector of the generated pursuer configuration and the utility feature vectors of the configurations on the Pareto front:

$$\mathcal{L}_{\text{Pareto}} = \min_{j \in \{1, \dots, m\}} \|\mathbf{y} - \mathbf{p}_j\| \quad (9)$$

where:

- $\mathbf{y}$ is the aggregated utility feature vector for the generated pursuer configuration, computed by averaging the utilities of all pursuers in the configuration:

$$\mathbf{y} = \frac{1}{n} \sum_{i=1}^n \left(\tilde{U}_{\text{distance}_i}, \tilde{U}_{\text{capture}_i}, \tilde{U}_{\text{heading}_i} \right)$$

where n is the number of pursuers.

- $\mathbf{p}_j$ is the utility feature vector of the j -th configuration on the Pareto front.
- m is the number of configurations on the Pareto front.

This loss function encourages the network to generate configurations whose aggregated utility vectors are close to those of the Pareto-optimal configurations.

5.2 Network Layers and Training

Our GCN processes the features of all pursuers in a configuration simultaneously, considering the interactions between pursuers through the graph structure. The input features for each pursuer node include:

- **Pursuer Position** (x, y): The initial positions of the pursuers.
- **Pursuer Velocity** (v_x, v_y): The initial velocities or headings of the pursuers.
- **Additional Features**: Such as initial utility estimates and capture radius or apollonious circle.

The edges in the graph represent connections between pursuers (PP edges), allowing the GCN to aggregate information from neighboring nodes and consider their relative positions and features.

The network outputs adjusted positions and velocities for all pursuers simultaneously, ensuring that the generated configuration

is coherent and optimized at the group level. By processing the entire configuration as a graph, the GCN accounts for interactions between pursuers, preventing issues such as overlapping positions and promoting effective spatial distribution.

Our network architecture and training parameters are detailed in Table 1. Training took place on 4,700 graphs from 12 game types (combinations of 1 to 5 evaders and 2 to 5 pursuers), effectively stacking the Pareto fronts of each game type to form the GFS. GD, GD+, IGD, and IGD+ were measured during training to assess how the generated graphs relate to our Pareto values. The hypervolume of the generated graphs was approximately 0.634 (see Appendix D). Hypervolume was not included in the loss function due to computational expense.

Although training ran for 150 epochs, the network converged earlier, as loss values decreased significantly over time. Each utility feature ranges from 0 to 1; thus, small changes in the GFS led to larger changes in actual results. Despite early convergence, extended training was necessary to fine-tune the network, as it was found that small changes in the GFS led to larger changes in actual results.

Table 1: Network Architecture and Loss Values, Training Parameters

(a) Network Architecture and Loss Values			(b) Training Parameters	
Layer	Nodes	Act.	Parameter	Value
Input	10	ReLU	Batch Size	1024
Hidden 1	64	ReLU	Learning Rate	1×10^{-4}
Hidden 2	32	ReLU	Optimizer	Adam
Hidden 3	64	ReLU	Weight Decay	1×10^{-2}
Output	2	Linear	Epochs	150
			Hardware	L4 GPU
Epoch		Loss		
1		0.145		
75		0.0247		
150		0.0234		

6 Experiments and Discussion

After training, our network generated 1000 configurations for each game type. These configurations were generated offline without prior knowledge of evader locations or numbers, then stored in a CSV, and supplied as initial pursuer locations as opposed to randomly placing them on a plane following a uniform distribution. To query our network, we passed in the amount of pursuers we would like it to generate configurations for (this could be thought of as the game type), as well as random input for the other features, meant to signify noise; this is meant to capture that fact that we have no observability about their velocities or locations. The network relies solely on pursuer-related inputs, producing positions and headings that are likely to perform well against unknown evader placements. This approach is distinct and fundamentally unique, as it enables the generation of strategically advantageous pursuer configurations without requiring real-time evader information.

Table 2: Survival Probabilities for 4 Pursuers vs. 2 Evaders (GCN vs. Random).

Time	GCN	Rand	Time	GCN	Rand
0	1.0000	1.0000	21	0.8722	0.9133
1	0.9991	0.9984	22	0.8619	0.9061
2	0.9981	0.9958	23	0.8511	0.8984
3	0.9967	0.9931	24	0.8398	0.8903
4	0.9943	0.9902	25	0.8279	0.8817
5	0.9909	0.9873	26	0.8154	0.8725
6	0.9865	0.9842	27	0.8022	0.8627
7	0.9812	0.9811	28	0.7881	0.8522
8	0.9753	0.9777	29	0.7732	0.8409
9	0.9691	0.9742	30	0.7572	0.8285
10	0.9626	0.9705	31	0.7398	0.8149
11	0.9559	0.9667	32	0.7210	0.7998
12	0.9488	0.9626	33	0.7002	0.7829
13	0.9415	0.9583	34	0.6772	0.7637
14	0.9339	0.9537	35	0.6511	0.7418
15	0.9260	0.9489	36	0.6208	0.7160
16	0.9179	0.9438	37	0.5843	0.6848
17	0.9094	0.9384	38	0.5385	0.6448
18	0.9007	0.9327	39	0.4753	0.5882
19	0.8915	0.9266	40	0.3635	0.4846
20	0.8821	0.9201	–	–	–

Note: Times 0–20 are in the left half, 21–40 in the right half. Survival probabilities estimated via Kaplan-Meier at each discrete time step.

We randomly sampled 10 batches of 100 configurations for each game type and tested them, comparing results with configurations generated using random placements. The results demonstrate that our network effectively learns generalizable configurations that improve performance in pursuit-evasion games, even without specific evader information.

6.1 Methodology

Hot starts are evaluated using a Monte Carlo analysis, aggregating results across numerous simulations. One can think of random starts as a control group and hot starts as the treatment group. Then the Kaplan-Meier estimator, a nonparametric metric from biostatistics [13, 16, 30], was used to estimate evader survival probabilities over time. Log-rank tests were performed to validate the statistical significance of our results.

For One Evader vs. Many Pursuers games, containment by calculating the area of the convex-hull formed by pursuers, checking if the evader was inside the hull using Delaunay triangulation was additionally measured. Further, conditional density plots were used to visualize this.

6.1.1 Many Evaders vs. Many Pursuers Results. Across all game types, hot starts reduced evader survival rates compared to random starts (see Appendix section G for full results). In just about every game configuration, the survival rate of evaders for strategies aided by hot starts dropped at a faster rate when compared to without them. Heatmaps revealed distinct movement patterns and activity areas for hot starts, mainly in the activeness of movement and area. Log-rank tests confirmed the statistical significance of our survival analysis, with, $p\text{-value} < .0001$ for all configurations.

6.1.2 One Evader vs. Many Pursuers Results. For One Evader vs. Many Pursuers games, the difference in survival rates between hot starts and random starts was less pronounced (see Appendix Section G for full results). Log-rank tests showed statistically significant differences, except in the 3 pursuers vs. 1 evader game. The heatmaps revealed similar movement patterns across starts, with differences mainly in movement frequency.

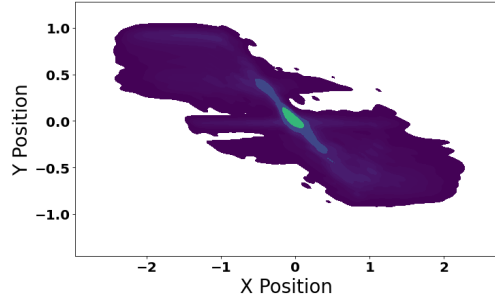
Several factors, including the diminishing returns of hot starts in high-pursuer-density scenarios, contribute to this outcome. The high density ensures quick evader interception, reducing the impact of initial configurations; essentially, since the ratio of pursuers to evaders is very high, an advantageous start has less of an impact on movement and the outcomes of the game.

6.2 Discussion

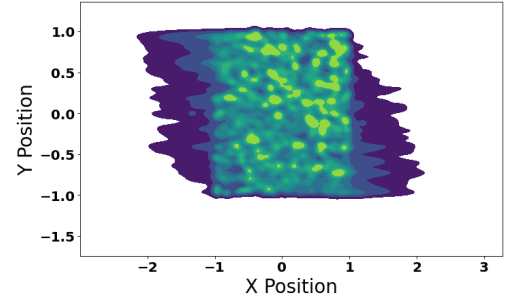
6.2.1 Many Evaders vs. Many Pursuers. Hot starts provided a clear advantage, as evader survival rates consistently dropped faster compared to random starts, as seen in figures 5a-b (see Table ?? for a direct example of 4 Pursuers vs. 2 Evaders games). The statistical significance of these results, supported by log-rank tests with $p\text{-values}$ near zero, indicates that hot starts substantially influence game outcomes, favoring pursuers. This highlights the efficacy of our method in capturing and leveraging the spatio-temporal dynamics of agents, allowing pursuers to gain a strategic upper hand.

Moreover, heatmaps revealed that hot starts naturally led to strategic partitioning of the environment, such as dividing the space into distinct regions (e.g., diagonal lines, crescent shapes, or "S" curves, see figure 1 for an example). This partitioning was not pre-programmed but emerged as the GCN learned latent configurations that strategically positioned pursuers from the outset. These emergent patterns ensured that pursuers were never entirely out of position, even in scenarios where random evader placement might suggest no clear advantage. The strategic placement resulted in enhanced capture rates, demonstrating the power of the GCN in discovering and generating effective configurations that maximize pursuit efficiency over many games.

6.2.2 One Evader vs. Many Pursuers. The survival analysis and heatmaps for one evader versus many pursuers differed notably from the many vs. many games. Although the visual differences in survival rates between hot starts and random starts were minimal, log-rank tests still indicated statistically significant results ($p\text{-value} < 0.05$) in all but one game scenario (in 3 pursuers vs. 1 evader games there was no statistical significance). This suggests that, while hot starts have a statistically measurable impact on evader survival, the practical effect is less pronounced due to the overwhelming number of pursuers, which rapidly neutralizes the advantage. High pursuer

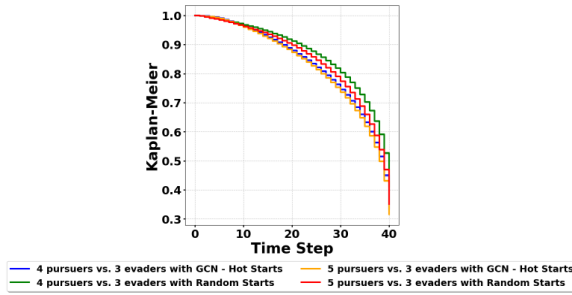


(a) 4 Pursuers vs. 2 Evaders GCN - Hot Starts heat map

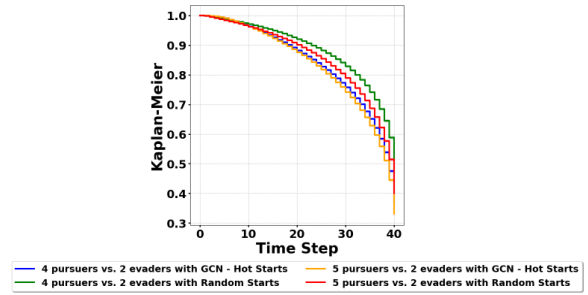


(b) 4 Pursuers vs. 2 Evaders Random Start heat map

Figure 4: Example Heat Maps for Many Pursuers vs. Many Evaders Games. The brighter the color, the more activity in that area of movement. See Appendix section G.4 for heatmaps for all Many Pursuers vs. Many Evaders Games.



(a) Survival Analysis of Game Simulations with 3 Evaders



(b) Survival Analysis of Game Simulations with 2 Evaders

Figure 5: Survival Analysis for Many Pursuers vs. Many Evaders Game Simulations. The x-axis is the time step and the y-axis is the value of the Kaplan-Meier Estimator.

density ensures that evaders are intercepted quickly, regardless of initial configuration, leading to similar containment outcomes in both hot and random starts.

6.2.3 Future Work and Limitations. This study serves as a proof of concept, establishing the foundational framework for our approach. Future work will concentrate on scaling the network, exploring additional game formats, expanding to larger environments, and applying our methodology to domains such as networked economics, robot swarms, and population initialization. Additionally, we are investigating the incorporation of Generative Adversarial Networks (GANs), Temporal Graph Networks, much more expressive GFS features, and human-in-the-loop systems similar to TacticAI.

While our evaluation spanned a diverse range of game types, limitations remain primarily due to the simplified environments chosen for public accessibility. As a result, the generalizability of hot starts to more complex or varied game scenarios is still an open question. Additionally, the current implementation of the Graph Feature Space relies on heuristics that, while effective for initial studies, lack the sophistication required for capturing all nuances of the underlying dynamics. Future work will focus on integrating the costs derived from the Hamilton-Jacobi-Isaacs (HJI) framework into the objective space, which we expect to yield a more robust performance metric. Furthermore, exploring the impact of

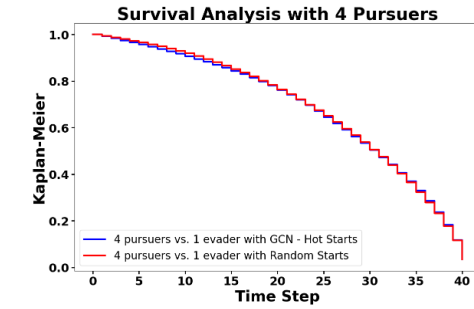
hot starts on computing open-loop equilibria represents a promising theoretical direction, potentially offering deeper insights into the dynamics of pursuit-evasion games.

7 Conclusion

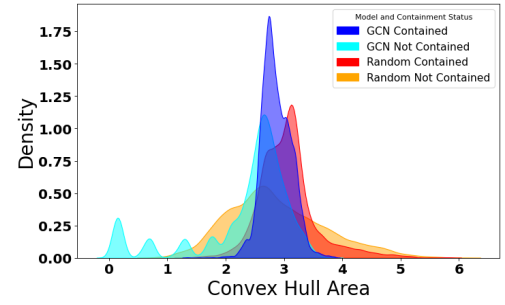
This paper presents a novel framework that addresses the problem of strategically placing pursuers when evader locations are unknown, using "hot starts." Hot starts are generated by applying differential game theory algorithms, tracking pursuer movements, and modeling configurations through graphs. By solving a multi-objective optimization problem to identify Pareto-optimal graphs, then proceed to train a GCN on them, and use the GCN to generate hot starts. Empirical evaluations demonstrate that these hot starts offer advantages in terms of faster evader capture, improved containment, and optimized agent movement. Statistical analysis, including log-rank tests, confirms the significance of these results. While this work serves as a proof of concept, it provides a promising foundation for future research.

Acknowledgments

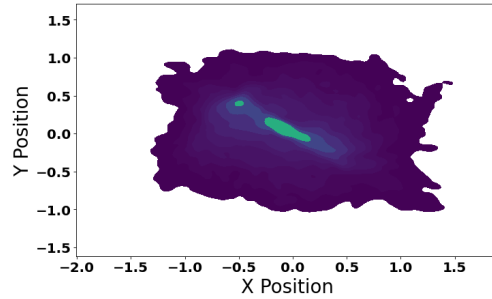
This work was generously supported by the Air Force Research Laboratory and the Hap Arnold Scholars Fellowship. Special thanks to Dr. Alex Von Moll and Dr. Isaac Weintraub for their guidance as well.



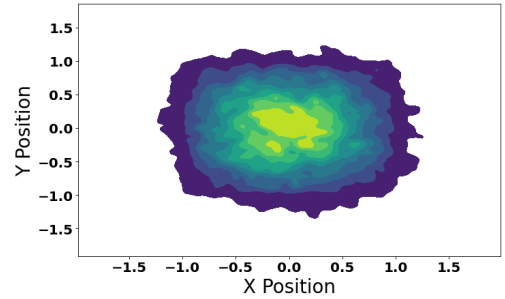
(a) 4 Pursuers vs. 1 Evader Survival Analysis



(b) 4 Pursuers vs. 1 Evader Conditional Density plot of containment



(c) 4 Pursuers vs. 1 Evader Hot Start heat map



(d) 4 Pursuers vs. 1 Evader Random Start heat map

Figure 6: Survival Analysis and movement analysis for 4 pursuers vs. 1 evader games. See Appendix section G.1 and G.2 for movement analysis and survival analysis tables for all 1 Evader vs. Many Pursuers games.

References

- [1] Jonathan Annas and Jing Xiao. 2009. Intelligent pursuit & evasion in an unknown environment. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 4899–4906.
- [2] Adonis Antoniadis, H Jin Kim, and Shankar Sastry. 2003. Pursuit-evasion strategies for teams of multiple agents with incomplete information. In *42nd IEEE International Conference on Decision and Control (IEEE Cat. No. 03CH37475)*, Vol. 1. IEEE, 756–761.
- [3] Efsthios Bakolas and Panagiotis Tsiotras. 2010. Optimal pursuit of moving targets using dynamic Voronoi diagrams. In *49th IEEE conference on decision and control (CDC)*. IEEE, 7431–7436.
- [4] Martino Bardi, Italo Capuzzo Dolcetta, et al. 1997. *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*. Vol. 12. Springer.
- [5] Tamer Başar and Geert Jan Olsder. 1998. *Dynamic noncooperative game theory*. SIAM.
- [6] J. Blank and K. Deb. 2020. pymoo: Multi-Objective Optimization in Python. *IEEE Access* 8 (2020), 89497–89509.
- [7] Elizabeth Bondi, Fei Fang, Mark Hamilton, Debarun Kar, Donnabell Dmello, Jongmoo Choi, Robert Hannaford, Arvind Iyer, Lucas Joppa, Milind Tambe, et al. 2018. Spot poachers in action: Augmenting conservation drones with automatic detection in near real time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [8] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMI Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [9] Tomonari Furukawa, Frederic Bourgault, Benjamin Lavis, and Hugh F Durrant-Whyte. 2006. Recursive Bayesian search-and-tracking using coordinated UAVs for lost targets. In *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006*. IEEE, 2521–2526.
- [10] Eloy Garcia and Shaunak D. Bopardikar. 2021. Cooperative Containment of a High-speed Evader. In *2021 American Control Conference (ACC)*. 4698–4703. <https://doi.org/10.23919/ACC50511.2021.9483097>
- [11] Eloy Garcia, David W Casbeer, Alexander Von Moll, and Meir Pachter. 2020. Multiple pursuer multiple evader differential games. *IEEE Trans. Automat. Control* 66, 5 (2020), 2345–2350.
- [12] C Giovannangeli, M Heymann, E Rivlin, and V Kordic. 2010. Pursuit-Evasion Games in Presence of Obstacles in Unknown Environments: towards an optimal pursuit strategy. *Cutting edge robotics* 2010 (2010), 47–80.
- [13] Manish Kumar Goel, Pardeep Khanna, and Jugal Kishore. 2010. Understanding survival analysis: Kaplan-Meier estimate. *International journal of Ayurveda research* 1, 4 (2010), 274.
- [14] Rufus Isaacs. 1969. Differential games: Their scope, nature, and future. *Journal of Optimization Theory and Applications* 3 (1969), 283–295.
- [15] Rufus Isaacs. 1999. *Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization*. Courier Corporation.
- [16] Edward L Kaplan and Paul Meier. 1958. Nonparametric estimation from incomplete observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481.
- [17] Borhan Kazimipour, Xiaodong Li, and A Kai Qin. 2014. A review of population initialization techniques for evolutionary algorithms. In *2014 IEEE congress on evolutionary computation (CEC)*. IEEE, 2585–2592.
- [18] Edward Keedwell, Mathieu Brevilliers, Lhassane Idoumghar, Julien Lepagnot, and Hojjat Rakhshani. 2018. A Novel Population Initialization Method Based on Support Vector Machine. In *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 751–756. <https://doi.org/10.1109/SMC.2018.00136>
- [19] David W King, Jason M Bindewald, and Gilbert L Peterson. 2018. Informal team assignment in a pursuit-evasion game. In *The Thirty-First International Flairs Conference*.
- [20] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [21] Pengdeng Li, Shuxin Li, Xinrun Wang, Jakub Cerny, Youzhi Zhang, Stephen McAleer, Hau Chan, and Bo An. 2024. Grasper: A Generalist Pursuer for Pursuit-Evasion Problems. *arXiv preprint arXiv:2404.12626* (2024).
- [22] Shuxin Li, Xinrun Wang, Youzhi Zhang, Wanqi Xue, Jakub Černý, and Bo An. 2023. Solving large-scale pursuit-evasion games using pre-trained strategies. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11586–11594.
- [23] Yang Li, Giovanni Carboni, Felipe Gonzalez, et al. 2019. Differential game theory for versatile physical human-robot interaction. *Nature Machine Intelligence* 1 (2019), 36–43. <https://doi.org/10.1038/s42256-018-0010-3>
- [24] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. *Neural Information Processing Systems (NIPS)* (2017).

- [25] Dejan Milutinović, David W. Casbeer, Alexander Von Moll, Meir Pachter, and Eloy Garcia. 2023. Rate of Loss Characterization That Resolves the Dilemma of the Wall Pursuit Game Solution. *IEEE Trans. Automat. Control* 68, 1 (2023), 242–256. <https://doi.org/10.1109/TAC.2021.3137786>
- [26] Ian M. Mitchell, Alexandre M. Bayen, and Claire J. Tomlin. 2005. A Time-Dependent Hamilton–Jacobi Formulation of Reachable Sets for Continuous Dynamic Games. *IEEE Trans. Automat. Control* 50, 7 (2005), 947–957. <https://doi.org/10.1109/TAC.2005.851439>
- [27] Igor Mordatch and Pieter Abbeel. 2017. Emergence of Grounded Compositional Language in Multi-Agent Populations. *arXiv preprint arXiv:1703.04908* (2017).
- [28] Stanley Osher and Ronald Fedkiw. 2003. *Level Set Methods and Dynamic Implicit Surfaces*. Applied Mathematical Sciences, Vol. 153. Springer. <https://doi.org/10.1007/b98879>
- [29] M V Ramana and Mangal Kothari. 2017. Pursuit-evasion games of high speed evader. *Journal of intelligent & robotic systems* 85 (2017), 293–306.
- [30] Jason T Rich, J Gail Neely, Randal C Paniello, Courtney CJ Voelker, Brian Nussenbaum, and Eric W Wang. 2010. A practical guide to understanding Kaplan-Meier curves. *Otolaryngology—Head and Neck Surgery* 143, 3 (2010), 331–336.
- [31] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.
- [32] Milind Tambe. 2011. *Security and game theory: algorithms, deployed systems, lessons learned*. Cambridge university press.
- [33] Alexander Von Moll, David W Casbeer, Eloy Garcia, and Dejan Milutinović. 2018. Pursuit-evasion of an evader by multiple pursuers. In *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 133–142.
- [34] Zhe Wang, Petar Veličković, Daniel Hennes, Nenad Tomašev, Laurel Prince, Michael Kaisers, Yoram Bachrach, Romuald Elie, Li Kevin Wenliang, Federico Piccinini, et al. 2024. TacticAI: an AI assistant for football tactics. *Nature communications* 15, 1 (2024), 1–13.
- [35] Isaac E Weintraub, Meir Pachter, and Eloy Garcia. 2020. An introduction to pursuit-evasion differential games. In *2020 American Control Conference (ACC)*. IEEE, 1049–1066.
- [36] Lily Xu, Shahrzad Gholami, Sara McCarthy, Bistra Dilkina, Andrew Plumptre, Milind Tambe, Rohit Singh, Mustapha Nsubuga, Joshua Mabonga, Margaret Driciru, et al. 2020. Stay ahead of poachers: Illegal wildlife poaching prediction and patrol planning under uncertainty with field test evaluations (short version). In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1898–1901.
- [37] Leiming Zhang, Amanda Prorok, and Subhrajit Bhattacharya. 2021. Pursuer assignment and control strategies in multi-agent pursuit-evasion under uncertainties. *Frontiers in Robotics and AI* 8 (2021), 691637.
- [38] Hongrui Zheng, Zhijun Zhuang, Stephanie Wu, Shuo Yang, and Rahul Mangharam. 2024. Bridging the Gap between Discrete Agent Strategies in Game Theory and Continuous Motion Planning in Dynamic Environments. *ArXiv abs/2403.11334* (2024). <https://api.semanticscholar.org/CorpusID:268513329>
- [39] Eckart Zitzler, Dimo Brockhoff, and Lothar Thiele. 2007. The hypervolume indicator revisited: On the design of Pareto-compliant indicators via weighted integration. In *Evolutionary Multi-Criterion Optimization: 4th International Conference, EMO 2007, Matsushima, Japan, March 5-8, 2007. Proceedings 4*. Springer, 862–876.